

STACK

1. STACK ARRAY IMPLEMENT

CODE :

```
#include <stdio.h>

int main()
{
    char s[100000];
    scanf("%s",s);
    int top=-1;
    char copy[100000];
    for(int i=0;s[i]!='\0';i++){
        if(s[i]=='(' || s[i]=='[' || s[i]=='{'){
            copy[++top]=s[i];
        }
        else if(s[i]==')' || s[i]==']' || s[i]=='}'){
            if(s[i]==(copy[top] + 1) || s[i]==(copy[top] + 2)){
                top--;
            }
        }
    }
    if(top==0){
        printf("Valid");
    }else{
        printf("Invalid");
    }
    return 0;
}
```

2. LIST IMPLEMENT

CODE :

```
#include <stdio.h>
#include <stdlib.h>
struct node{
    char data;
    struct node *next;
}*head=NULL;

void append(char a){
    struct node newnode=(struct node)malloc(sizeof (struct node));
```

```

    newnode->data=a;
    newnode->next=head;
    head=newnode;

}
void pop(){
    struct node *temp=head;
    head=head->next;
    free(temp);
}
void check(){
    if(head==NULL){
        printf("Valid");
    }
    else{
        printf("Invalid");
    }
}
int main()
{
    char s[100000];
    scanf("%s",s);
    for(int i=0;s[i]!='\0';i++){
        if(s[i]=='(' || s[i]=='[' || s[i]=='{'){
            append(s[i]);
        }
        else if(s[i]==')' || s[i]==']' || s[i]=='}'){
            if(head==NULL){
                printf("Invalid");
                return 0;
            }
            if(s[i]==(head->data + 1) || s[i]==(head->data + 2)){
                pop();
            }
        }
    }
    check();

    return 0;
}

```

3. DELETE ARRAY IMPLEMENT CODE :

```
#include <stdio.h>
```

```
int main()
```

```

{
    char s[100000];
    scanf("%s",s);
    int top=-1;
    char copy[100000];
    for(int i=0;s[i]!='\0';i++){
        copy[++top]=s[i];
        if(top>0 && copy[top]==copy[top-1]){
            top-=2;
        }
    }
    if(top==0){
        printf("EMPTY");
    }else{
        for(int k=0;k<=top;k++){
            printf("%c",copy[k]);
        }
    }
    return 0;
}

```

4. DELETE LIST IMPLEMENT

CODE :

```

#include <stdio.h>
#include<stdlib.h>
#include<string.h>
struct node{
    char data;
    struct node *next;
}*head=NULL;

void append(char a){
    struct node newnode=(struct node)malloc(sizeof(struct node));
    newnode->data=a;
    newnode->next=head;
    head=newnode;
}

void pop(){
    struct node *temp=head;
    head=head->next;
    free(temp);
}

void check(){
    if(head==NULL){
        printf("EMPTY");
    }
}

```

```

    }
    else{
        char g[10000];
        int y=0;
        struct node *temp=head;
        while(temp!=NULL){
            g[y]=temp->data;
            temp=temp->next;
            y++;
        }
        for(int i=strlen(g)-1;i>=0;i--){
            printf("%c",g[i]);
        }
    }
}
int main()
{
    char s[100000];
    scanf("%s",s);
    for(int i=0;s[i]!='\0';i++){

        if(head!=NULL && s[i]==head->data){
            pop();
        }
        else{
            append(s[i]);
        }
    }
    check();

    return 0;
}

```

5. REVERSE

CODE :

```

#include <stdio.h>
#include<stdlib.h>
#include<string.h>
struct node{
    char data;
    struct node *next;
}*head=NULL;

```

```

void append(char a){
    struct node newnode=(struct node)malloc(sizeof(struct node));
    newnode->data=a;
}

```

```

newnode->next=head;
head=newnode;

}
void display(){
    struct node *temp=head;
    while(temp!=NULL){
        printf("%c",temp->data);
        temp=temp->next;
    }
}

int main()
{
    char s[100000],x=' ';
    scanf("%[^\\n]",s);
    int k;
    for(int i=0;s[i]!='\\0';i++){
        if(s[i]==' ' || i==strlen(s)-1){
            k=i-1;
            if(i==strlen(s)-1){
                k=i;
            }
            while(k>=0){
                if(s[k]==' '){
                    if(i!=strlen(s)-1){
                        append(x);
                    }
                    break;
                }
                append(s[k]);
                if(k==0){
                    append(x);
                }
                k--;
            }
        }
    }
    display();

    return 0;
}

```

6. NEXT GREATER ELEMENT

CODE :

```

#include <stdio.h>

int main()
{
    int n,x=0;
    scanf("%d",&n);
    int arr[n];
    for(int i=n-1;i>=0;i--){
        scanf("%d",&arr[i]);
    }
    for(int i=n-1;i>=0;i--){
        if(i==0){
            arr[i]=-1;
            break;
        }
        for(int j=i-1;j>=0;j--){
            if(arr[i]<arr[j]){
                arr[i]=arr[j];
                x++;
                break;
            }
        }
        if(x==0){
            arr[i]=-1;
        }
        x=0;
    }
    for(int k=n-1;k>=0;k--){
        printf("%d ",arr[k]);
    }
    return 0;
}

```

7. MIN STACK OPERATIONS

CODE :

```

#include <stdio.h>
#include<stdlib.h>
#include<string.h>
struct node{
    int data;
    struct node *next;
}*head=NULL;

void append(int a){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=a;
    newnode->next=head;
    head=newnode;
}

```

```

}
void pop(){
    struct node *temp=head;
    head=head->next;
    free(temp);
}
void min(){
    struct node *temp=head;
    if(head==NULL){
        printf("EMPTY\n");
    }else{
        int m=head->data;
        while(temp!=NULL){
            if(temp->next!=NULL && temp->data > temp->next->data){
                m=temp->next->data;
            }
            temp=temp->next;
        }printf("%d\n",m);
    }
}
int main()
{
    int n,a;
    scanf("%d",&n);
    char s[100];
    for(int i=0;i<n;i++){
        scanf("%s",s);
        if(s[1]=='U'){
            scanf("%d",&a);
            append(a);
        }else if(s[1]=='O'){
            if(head==NULL){
                printf("EMPTY");
            }else{
                pop();
            }
        }
        else if(s[1]=='I'){
            min();
        }
    }
    return 0;
}

```

8. POSTFIX EXPRESSION EUATION
CODE :

```

#include <stdio.h>
#include<stdlib.h>
#include<string.h>
struct node{
char data;
struct node *next;
}*head=NULL;
void append(char a){
struct node *newnode=(struct node*)malloc(sizeof(struct node));
newnode->data=a;
newnode->next=head;
head=newnode;
}
void pop(){
struct node *temp=head;
head=head->next;
free(temp);
}
int main()
{
char s[100000];
scanf("%s",s);
int a;
for(int i=0;s[i]!='\0';i++){
if(s[i]=='+' || s[i]=='-' || s[i]=='*' || s[i]=='/'){
if(head!=NULL &&head->next!=NULL && (head->next->data!='+' || head->next->data!='-' || head->next->data!='*' || head->next->data!='/')){
if(s[i]=='+'){
a=(head->data - '0')+(head->next->data- '0');
}
else if(s[i]=='-'){
if((head->data - '0')>(head->next->data - '0')){
a=(head->data - '0')-(head->next->data- '0');
}else{
a=(head->next->data - '0')-(head->data- '0');
}
}
else if(s[i]=='*'){
a=(head->data - '0')*(head->next->data- '0');
}
else if(s[i]=='/'){
if((head->data - '0')>(head->next->data - '0')){
a=(head->data - '0')/(head->next->data- '0');
}else{
a=(head->next->data - '0')/(head->data- '0');
}
}
}
}
}

```

```

        }
        pop();
        pop();
        append(a+'0');
    }
    }else{
        append(s[i]);
    }
}
printf("%d",a);

return 0;
}

```

9. BASEBALL OPERATION

CODE :

```

#include <stdio.h>
#include<stdlib.h>
#include<string.h>
struct node{
    int data;
    struct node *next;
}*head=NULL;

void append(int a){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=a;
    newnode->next=head;
    head=newnode;
}

void pop(){
    struct node *temp=head;
    if(head!=NULL){
        head=head->next;
        free(temp);
    }
}

void ans(){
    int sum=0;
    struct node*temp=head;
    while(temp!=NULL){
        sum+=(temp->data);
        temp=temp->next;
    }
    printf("%d",sum);
}

```

```

int main()
{
    int n;
    scanf("%d",&n);
    char s[n][10];
    for(int i=0;i<n;i++){
        scanf(" %s",s[i]);
    }
    int a;
    for(int i=0;i<n;i++){
        if(strcmp(s[i],"C")==0){
            pop();
        }else if(strcmp(s[i],"D")==0){
            if(head!=NULL){
                int x=(head->data)*2;
                append(x);
            }
        }else if(strcmp(s[i],"+")==0){
            if(head!=NULL && head->next!=NULL){
                a=(head->data)+(head->next->data);
                append(a);
            }
        }else{
            append(atoi(s[i]));
        }
    }
    ans();
    return 0;
}

```

10. REMOVE STAR FROM STRING
CODE :

```

#include <stdio.h>
#include<stdlib.h>
#include<string.h>
struct node{
    char data;
    struct node *next;
}*head=NULL;
void append(char a){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=a;
    newnode->next=head;
    head=newnode;
}

```

```

void pop(){
struct node *temp=head;
if(head!=NULL){
head=head->next;
free(temp);
}
}
void ans(){
if(head==NULL){
printf("EMPTY");
}else{
    struct node *temp=head;
    char str[1000];
    int i=0,c=0;
    struct node *temp1=head;
    while(temp1!=NULL){
        c++;
        temp1=temp1->next;
    }
    while(temp!=NULL){
        str[i]=temp->data;
        temp=temp->next;
        i++;
    }
    for(int j=c-1;j>=0;j--){
        printf("%c",str[j]);
    }
}
}
int main()
{
    char s[1000000];
    scanf("%s",s);
    int a;
    for(int i=0;s[i]!='\0';i++){
        if(s[i]=='*'){
            pop();
        }
        else{
            append(s[i]);
        }
    }
    ans();

    return 0;
}

```

11. NEXT WARMER DAY FORECAST

CODER :

```
#include <stdio.h>
#include<stdlib.h>
#include<string.h>

int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int ans[n],stack[n],top=-1,k=0;
    for(int i=0;i<n;i++){
        if(top==-1){
            stack[++top]=i;
        }else{
            while(top!=-1 && arr[i]>arr[stack[top]]){
                ans[stack[top]]=i-stack[top];
                top--;
            }
            stack[++top]=i;
        }
    }
    ans[n-1]=0;
    for(int j=0;j<n;j++){
        printf("%d ",ans[j]);
    }
    return 0;
}
```

12. Asteroid Collision Simulator

CODE :

```
#include <stdio.h>
#include<stdlib.h>
#include<string.h>

int main()
{
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
```

```

    }
    int ans[n],top=-1,k=0;
    for(int i=0;i<n;i++){
        if(top==-1){
            ans[++top]=arr[i];
        }else{
            if((ans[top]<0 && arr[i]>0) ||
(ans[top]>0&&arr[i]>0) || (ans[top]<0&&arr[i]<0)){
                ans[++top]=arr[i];
            }else{
                int x=1;
                while(top!=-1 && ans[top]>0 && arr[i]<0){
                    int a=-1*arr[i];
                    if(ans[top]<a){
                        top--;
                        continue;
                    }else if(ans[top]>a){
                        x=0;
                        break;
                    }
                    else if(ans[top]==a){
                        top--;
                        x=0;
                        break;
                    }
                }
                if((x==1)&&(top==-1 || ans[top]<0)){
                    ans[++top]=arr[i];
                }
            }
        }
    }

    for(int j=0;j<=top;j++){
        printf("%d ",ans[j]);
    }
    return 0;
}

```